

Un solo cancello davanti a tutto.

Xal-Tor-Ka autentica *chi* entra, decide *dove* va, mette il TLS — e, se vuoi, ospita i tuoi siti in container isolati. Tutto da un pannello solo, dietro un unico punto esposto.

1 punto esposto

TLS automatico + CA interna

OIDC · LDAP / AD · 2FA

Hosting Docker isolato

Chi gestisce più servizi web ripete ovunque le stesse cose: login, HTTPS, chi-può-vedere-cosa, e la fatica di mettere online un sito nuovo. **Xal-Tor-Ka centralizza tutto questo davanti:** un servizio dietro il gate non deve più pensare né all'autenticazione né ai certificati — li mette il gate, in modo uniforme e auditabile.

Un solo login, davanti a tutto

Autenticazione centralizzata per ogni servizio dietro il gate. Le fonti d'identità sono componibili: utenti locali con 2FA (TOTP), provider **OIDC** (Google, Microsoft, qualsiasi issuer standard), e **LDAP / Active Directory**. Un servizio protetto non vede mai una richiesta non autenticata.

● ● ● /login

EN

Sign in

Email

Password

Continue

Xal-Tor-Ka · beta0.6

Reverse-proxy con regole per servizio

Ogni servizio è un backend con una regola d'accesso:

public aperto, authenticated richiede il login, whitelist solo IP consentiti. Si gestisce dal pannello — host pubblico, upstream interno, regola — senza toccare file a mano.

/admin/servizi

Xal-Tor-Ka

ADMINISTRATION

beta0.6

Services

Docker

Providers

Users

Monitoring

TLS

Hosting

Dashboard

EN

Reverse-proxy services

Services from config.json are read-only (infrastructure). Those added here live in services.json.

SERVICE	HOST / UPSTREAM	RULE	IP ALLOW
Demo	demo.localhost http://demo:80	PUBLIC	edit Manage SSL disable delete
myrules-builder-backend-1	myrules-builder-backend-1.localhost http://host.docker.internal:8765	PUBLIC	edit Manage SSL enable delete
myrules	myrules.localhost http://host.docker.internal:5173	AUTHENTICATED	edit Manage SSL disable delete
example-com/httpdocs (hosting)	example.com http://example-com.site:8080	PUBLIC	edit Manage SSL disable delete
example-com/app (hosting)	app.example.com http://example-com-app.site:8080	PUBLIC	edit Manage SSL disable delete
Demo service	demo.example.com http://demo:80	AUTHENTICATED	edit Manage SSL disable delete

Add proxied service

id

name

host

path

rule

upstream

public url

www

IP allow (CIDR, opt.)

add

public: open to everyone. authenticated: users with a valid session (2FA). whitelist: only explicitly authorized users (implies authentication); administrators always have access (override).

External links

Tiles in the dashboard, not proxied.

NAME	URL	VISIBILITY
Documentazione	https://example.com/docs	PUBLIC

Add link

id

name

public url

description

add

public

Certificati ovunque, senza sbatti

Let's Encrypt automatico per gli host pubblici, e una **CA interna** scaricabile per i servizi LAN/dev senza DNS. Il certificato segue il servizio: pubblici un host, poi lo emetti con un clic. I sottodomini stanno annidati sotto il dominio padre.

/admin/tls

Xal-Tor-Ka

ADMINISTRATION

beta0.6

Services

Docker

Providers

Users

Monitoring

TLS

Hosting

Dashboard

EN

Certificates

Issue and renew TLS certificates for the served hosts. Let's Encrypt (ACME) needs the host to resolve publicly to this gateway with port 80 reachable; use the internal CA for LAN/dev hosts without public DNS.

HOST	SOURCE	EXPIRY	STATUS				
demo.localhost	—	—	MISSING	☐ www.	issue LE	self-signed	delete
myrules.localhost	—	—	MISSING	☐ www.	issue LE	self-signed	delete
example.com	internal CA	2028-10-20	VALID	☐ www.	issue LE	self-signed	delete
↳ app.example.com	internal CA	2028-10-20	VALID	☐ www.	issue LE	self-signed	delete
↳ demo.example.com	—	—	MISSING	☐ www.	issue LE	self-signed	delete

Internal CA

Download internal CA

Install this CA on client devices to trust the self-signed certificates.

ACME account email: — set tls.acme.email in config.json (Let's Encrypt expiry notices)

Siti in Docker isolati, creati dal pannello

Oltre a proxare servizi esistenti, Xal-Tor-Ka crea siti da zero: ogni sito è un **utente di sistema isolato** con uno o più **vhost**, ciascuno nella sua Docker (NGINX + PHP-FPM, Apache, statico, Node, o compose custom). DB condivisi MySQL/PgSQL con utenti isolati, SFTP/SSH in chroot con chiavi.

● ● ● /admin/hosting

Una home per chi entra

Il catalogo dei servizi visibili è anche la home dell'utente autenticato: proxy interni e link esterni, ognuno con la sua etichetta di stato.

● ● ● /listing

Salute, audit, controllo remoto

Health dei backend in tempo reale, log di audit degli accessi, whitelist IP per l'admin, e notifiche/controllo remoto opzionali via Telegram/email — log di sistema a distanza con comandi vettati.

● ● ● /admin/monitoring

Perché ci si può fidare

- ▶ **Un solo punto esposto**

NGINX davanti, gate Go interno. I servizi stanno dietro, non sulla rete pubblica.

- ▶ **Admin blindato**

Area di amministrazione dietro whitelist IP + sessione. Ogni azione è auditata.

- ▶ **Agente host vettato**

Le operazioni privilegiate passano per un insieme fisso di script root-owned **non iniettabili**: i parametri arrivano come variabili d'ambiente, mai come shell.

- ▶ **Segreti fuori dal codice**

Repo pubblico, nessuna credenziale versionata. Configurazione dichiarativa e riproducibile.

COME SI FA

Le cose più frequenti, in un gesto

Metti auth + HTTPS davanti a un servizio

Services → Add backend · poi TLS → emetti il cert

Crea un sito nuovo

Hosting → New site: dominio + stack → Create & start

Aggiungi un sottodominio

scheda sito → + Add vhost → Publish

SSO aziendale

Providers (OIDC) · oppure LDAP / Active Directory

Accesso ai file del sito

scheda sito → SSH keys (SFTP in chroot)

Chi può entrare nell'admin

Whitelist IP admin

Xal-Tor-Ka

Software di **SFS.it**. Screenshot da un'installazione dimostrativa con dati di esempio (example.com).